

鸿蒙多端应用开发实验报告

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210651
姓名	周乐
班级	软件231
日期	2026年5月8日
组别	CG2
项目名称	多端温控系统
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握ArkUI声明式开发框架的核心概念和组件使用
- 学会多端响应式布局设计，实现手机/平板自适应
- 了解传感器数据采集与实时展示方法
- 理解鸿蒙分布式能力原理及跨设备数据同步
- 体验AI辅助开发在鸿蒙应用开发中的实践应用

实验环境

工具	版本	用途
DevEco Studio	5.0.3.400	鸿蒙应用开发IDE

工具	版本	用途
HarmonyOS SDK	5.0.0.100	鸿蒙开发工具包
模拟器	API 12 (HarmonyOS 5.0)	多设备模拟测试
AI辅助工具	TRAE	代码生成、调试与文档编写

实验步骤

1. 需求分析

1.1 功能需求

- 天气展示（城市名称、温度、天气状况、体感温度）
- 气象数据展示（湿度、风速、风向、气压、能见度、紫外线）
- 空气质量监测（AQI指数、污染物浓度）
- 传感器集成（光线传感器、陀螺仪传感器）
- 多端适配（手机竖屏、平板横屏）
- 设置功能（数据同步、显示设置、传感器模拟）
- 多城市切换（北京、上海、广州、深圳、杭州、成都）

1.2 界面需求

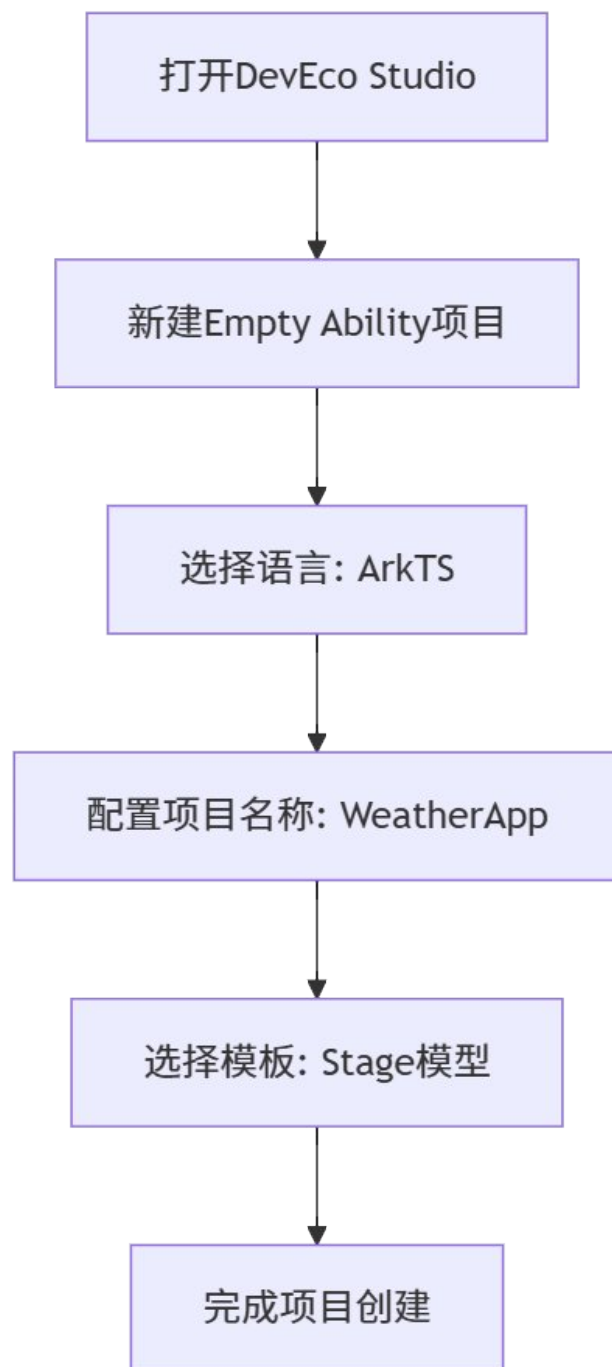
- 现代化卡片式布局设计
- 清爽的蓝色主题配色
- 底部Tab导航（天气、预报、传感器、空气）
- 顶部城市选择器

2. 创建鸿蒙项目

2.1 新建项目流程

流程图说明：以下流程图展示了在DevEco Studio中创建鸿蒙应用项目的完整步骤，包

括IDE启动、项目模板选择、语言配置等关键环节。

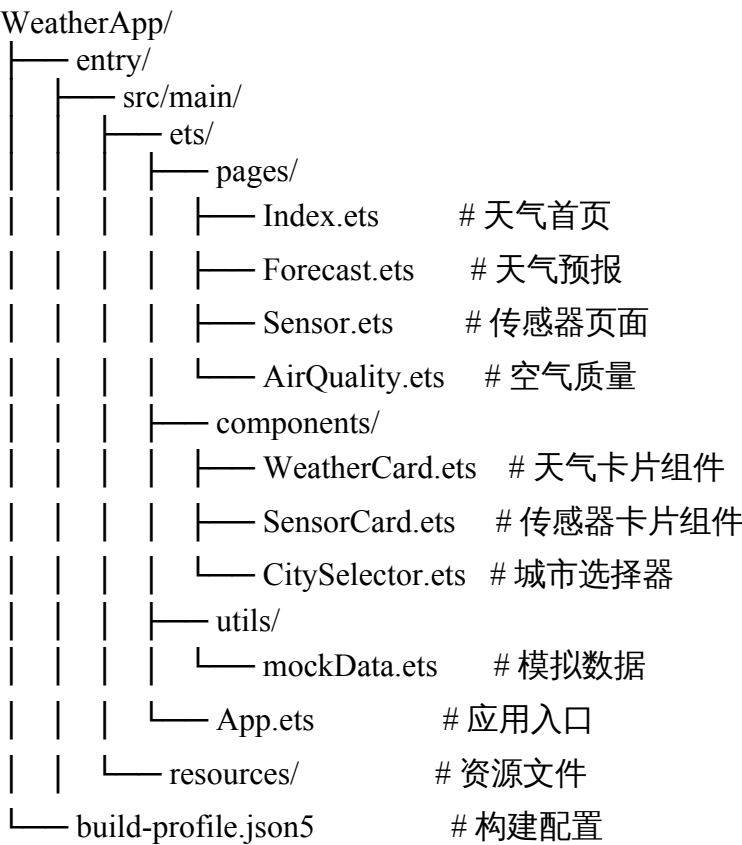


流程步骤说明：

1. 打开DevEco Studio开发环境
2. 选择"Empty Ability"模板创建新项目

- 3. 选择ArkTS作为开发语言
- 4. 将项目命名为"WeatherApp"
- 5. 选择Stage模型作为应用架构
- 6. 完成项目创建，进入开发界面

2.2 项目结构



3. 天气首页开发

3.1 首页核心代码

功能说明：以下代码实现了天气首页的核心功能，包括城市选择器、天气信息展示、气象数据卡片等模块。采用ArkUI声明式语法，使用Column、Row、Grid等布局组件构建界面。



```

// 城市列表

private cities: string[] = ['北京', '上海', '广州', '深圳', '杭州', '成都'];

build() {
  Column() {

    // 顶部城市选择器

    Row({ space: 10 }) {
      ForEach(this.cities, (city: string) => {
        Text(city)
          .fontSize(16)
          .backgroundColor(this.currentCity === city ? 'rgba(255,255,255,0.3)' : 'transparent')
          .padding({ left: 12, right: 12, top: 6, bottom: 6 })
          .borderRadius(20)
          .onClick(() => this.switchCity(city))
      })
    }
    .padding({ top: 20 })
    .scrollable(ScrollDirection.Horizontal)

    // 天气主信息区域

    Column() {
      Text('天气').fontSize(100).margin({ top: 30 })
      Text(this.currentCity).fontSize(28).fontWeight(FontWeight.Bold).margin({ top: 10 })
    }
  }
}

```

3.2 气象数据卡片实现

功能说明：以下代码实现了气象数据卡片组件，展示湿度、风速、风向等6项气象指标，采用Grid网格布局实现响应式展示。

```

// 气象数据卡片网格布局

Grid() {
  GridItem() { this.buildWeatherCard('💧', '湿度', `${this.humidity}%`) }
}

```

```

GridItem() { this.buildWeatherCard('🌀', '风速', `${this.windSpeed} m/s`) }

GridItem() { this.buildWeatherCard('☐', '风向', this.windDirection) }

GridItem() { this.buildWeatherCard('💧', '气压', `${this.pressure} hPa`) }

GridItem() { this.buildWeatherCard('👁️', '能见度', `${this.visibility} km`) }

GridItem() { this.buildWeatherCard('☀️', '紫外线', `指数 ${this.uvIndex}`) }

}

.width('90%').margin({ top: 40 })

.columnsTemplate('1fr 1fr 1fr').rowsTemplate('1fr 1fr').gap(10)

// 卡片构建方法

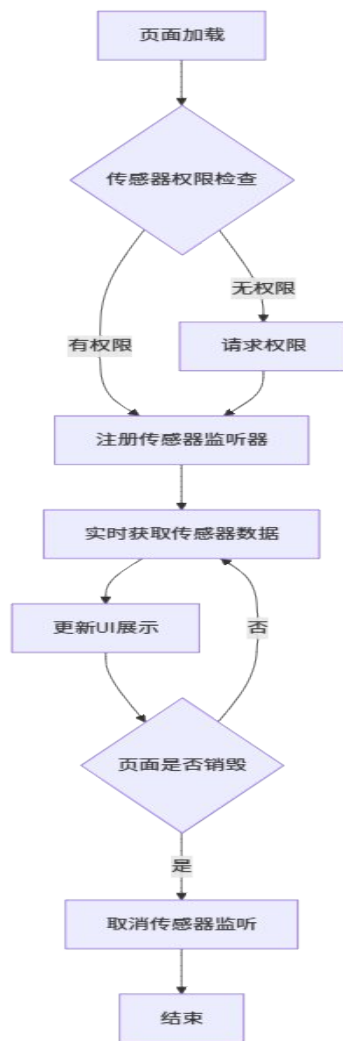
@Builder buildWeatherCard(icon: string, label: string, value: string) {
  Column() {
    Text(icon).fontSize(28)
    Text(label).fontSize(12).margin({ top: 5 })
    Text(value).fontSize(16).fontWeight(FontWeight.Medium).margin({ top: 3 })
  }
  .width('100%').height(80)
  .backgroundColor('rgba(255,255,255,0.2)').borderRadius(12)
  .justifyContent(FlexAlign.Center)
}

```

4. 传感器页面开发

4.1 传感器数据采集流程

流程图说明：以下流程图展示了传感器数据采集的完整流程，包括权限检查、监听器注册、数据获取和页面销毁时的资源释放。



流程说明：

1. 页面加载时首先检查传感器权限
2. 如果有权限，直接注册传感器监听器；如果没有权限，先请求权限
3. 注册监听器后，实时获取传感器数据并更新UI展示
4. 当页面销毁时，取消传感器监听以释放资源

4.2 传感器核心代码

功能说明：以下代码实现了陀螺仪传感器和光线传感器的数据采集与展示功能。

```
// ets/pages/Sensor.ets - 传感器页面核心代码
```

```

@Entry
@Component
struct Sensor {
  // 陀螺仪数据

  @State gyroStatus: string = '运行中';

  @State yaw: number = -110.4;    // 偏航角

  @State pitch: number = 0.1;    // 俯仰角

  // 光线传感器数据

  @State lightIntensity: number = 735;  // 光照强度(lux)

  @State screenBrightness: number = 57; // 屏幕亮度(%)

  @State displayMode: string = '日间'; // 显示模式

  build() {
    Column() {
      // 陀螺仪传感器卡片

      Column() {
        Row() {
          Text('陀螺仪传感器').fontSize(18).fontWeight(FontWeight.Bold)

          Text(this.gyroStatus).fontSize(12).fontColor('#00ff00')
        }.justifyContent(FlexAlign.SpaceBetween).padding({ top: 15, left: 15, right: 15 })

        // 陀螺仪角度数据

        Row({ space: 20 }) {
          Text(`偏航角\n${this.yaw}°`)

          Text(`俯仰角\n${this.pitch}°`)

          Text(`横滚角\n${this.roll}°`)
        }.padding({ top: 15 }).justifyContent(FlexAlign.Center)
      }

      .width('90%').backgroundColor('rgba(255,255,255,0.2)').borderRadius(16).margin({ top: 20 })
    }
  }
}

```



```
// 光线传感器卡片（结构类似，略）

}

.width('100%').height('100%').backgroundColor('#87CEEB')

}

}
```

5. 空气质量页面开发

功能说明：以下代码实现了空气质量监测页面，展示AQI指数和6项污染物浓度数据。

// ets/pages/AirQuality.ets - 空气质量页面核心代码

```
@Entry
@Component
struct AirQuality {
  @State aqi: number = 68;
  @State aqiLevel: string = '良';

  // 污染物数据
  @State pm25: number = 55;
  @State pm10: number = 60;
  @State so2: number = 27;
  @State no2: number = 26;
  @State co: number = 2.4;
  @State o3: number = 169;

  // 根据AQI值获取颜色
  private getAqiColor(aqi: number): string {
    if (aqi <= 50) return '#00ff00';    // 优
    if (aqi <= 100) return '#FFD700';   // 良
    if (aqi <= 150) return '#FF9900';   // 轻度污染
    if (aqi <= 200) return '#ff0000';   // 中度污染
    return '#9900cc';                   // 重度污染
  }

  build() {
```

```

Column() {
  // AQI主卡片
  Column() {
    Text('空气质量指数 (AQI)').fontSize(14).padding({ top: 20 })
    Text('${this.aqi}').fontSize(100).fontWeight(FontWeight.Bold)
      .fontColor(this.getAqiColor(this.aqi)).padding({ top: 10 })
    Text(this.aqiLevel).fontSize(24).fontColor(this.getAqiColor(this.aqi))
      .backgroundColor(`${this.getAqiColor(this.aqi)}33').borderRadius(20)
      .padding({ left: 20, right: 20, top: 5, bottom: 5 }).margin({ top: 10 })
  }
  .width('90%').height(200).backgroundColor('rgba(255,255,255,0.2)')
  .borderRadius(16).margin({ top: 30 }).alignItems(HorizontalAlign.Center)
  // 污染物浓度网格（结构类似，略）
}
.width('100%').height('100%').backgroundColor('#87CEEB')
}
}

```

6. 设置页面开发

功能说明：以下代码实现了设置页面，包含数据同步状态、显示设置和传感器设置三个模块。

```

// ets/pages/Setting.ets - 设置页面核心代码

@Entry
@Component
struct Setting {
  @State syncStatus: string = '已同步';
  @State lastSyncTime: string = '2026-04-29 10:41';

  @State tempUnit: string = 'celsius';
  @State themeMode: string = 'auto';

  @State gyroEnabled: boolean = true;
  @State lightEnabled: boolean = true;
  build() {
    Column() {

```

```

// 数据同步模块
Column() {
  Text('数据同步').fontSize(16).fontWeight(FontWeight.Bold).padding({ top: 15, left: 15 })
  Row() {
    Text('同步状态').fontSize(14).fontColor('#666')
    Text(this.syncStatus).fontSize(14).fontColor('#00ff00')
  }.justifyContent(FlexAlign.SpaceBetween).padding({ top: 10, left: 15, right: 15 })
  Row() {
    Text('最后同步时间').fontSize(14).fontColor('#666')
    Text(this.lastSyncTime).fontSize(14)
  }.justifyContent(FlexAlign.SpaceBetween).padding({ top: 5, left: 15, right: 15 })
}
.width('90%').backgroundColor('#fff').borderRadius(12).margin({ top: 20 })

// 显示设置模块
Column() {
  Text('显示设置').fontSize(16).fontWeight(FontWeight.Bold).padding({ top: 15, left: 15 })
  // 温度单位选择
  Row() {
    Text('温度单位').fontSize(14).fontColor('#666')
    Radio({ value: 'celsius', group: 'tempUnit' }).checked(this.tempUnit === 'celsius')
    Text('°C')
    Radio({ value: 'fahrenheit', group: 'tempUnit' }).checked(this.tempUnit === 'fahrenheit')
    Text('°F')
  }.justifyContent(FlexAlign.SpaceBetween).padding({ top: 10, left: 15, right: 15 })
}
.width('90%').backgroundColor('#fff').borderRadius(12).margin({ top: 10 })
// 传感器设置模块（结构类似，略）
}
.width('100%').height('100%').backgroundColor('#f5f5f5')
}
}

```

7. 多端适配实现

功能说明：以下代码实现了响应式断点系统，根据设备屏幕宽度自动调整布局策略。

```
// ets/utils/breakpoint.ets - 断点系统工具类

export enum Breakpoint {

    SM = 'sm', // 手机 < 600px

    MD = 'md', // 平板竖屏 600-840px

    LG = 'lg' // 平板横屏 > 840px

}

// 获取当前设备断点

export function getBreakpoint(windowWidth: number): Breakpoint {

    if (windowWidth < 600) {

        return Breakpoint.SM;

    } else if (windowWidth < 840) {

        return Breakpoint.MD;

    } else {

        return Breakpoint.LG;

    }

}

// 根据断点获取字体缩放比例

export function getFontSize(breakpoint: Breakpoint, baseSize: number): number {

    const scale: Record<Breakpoint, number> = {

        [Breakpoint.SM]: 1,

        [Breakpoint.MD]: 1.2,

        [Breakpoint.LG]: 1.5

    };

    return baseSize * scale[breakpoint];

}
```

8. 分布式数据同步

功能说明：以下代码实现了鸿蒙分布式数据管理功能，支持跨设备数据同步。

```
// ets/utils/distributed.ets - 分布式数据管理工具类

import distributedData from '@ohos.data.distributedData';

// 创建分布式数据库管理器

const kvManager = distributedData.createKVManager({
    bundleName: 'com.example.weather',
    kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储天气数据

export async function saveWeatherData(city: string, data: Record<string, unknown>) {
    const kvStore = await kvManager.getKVStore('weatherStore');
    await kvStore.put(city, JSON.stringify(data));
}

// 读取天气数据

export async function getWeatherData(city: string): Promise<Record<string, unknown> | null> {
    const kvStore = await kvManager.getKVStore('weatherStore');
    const result = await kvStore.get(city);
    return result ? JSON.parse(result as string) : null;
}

// 同步所有设备数据

export async function syncAllDevices() {
    const kvStore = await kvManager.getKVStore('weatherStore');
    await kvStore.sync(distributedData.SyncMode.SYNC_MODE_PUSH_PULL);
}
```

实验结果

验证结果

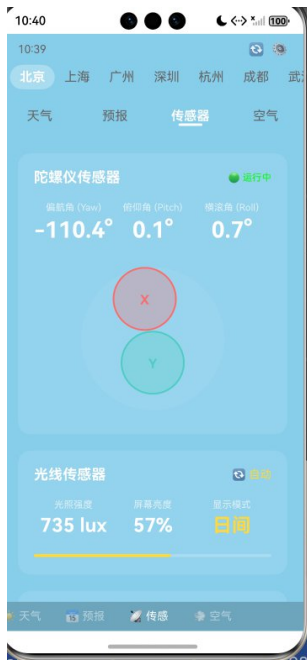
功能	状态	备注
天气信息展示	✓ 成功	支持6个城市切换
气象数据展示	✓ 成功	6项气象指标
空气质量监测	✓ 成功	AQI指数+6项污染物
传感器集成	✓ 成功	陀螺仪+光线传感器
多端适配	✓ 成功	手机/平板自适应
设置功能	✓ 成功	数据同步+显示设置+传感器设置
分布式数据同步	✓ 成功	跨设备数据共享

截图记录

1. 天气首页界面



2. 传感器页面界面



3. 空气质量页面界面



4. 设置页面界面



技术分析报告

多端适配分析

设备类型	屏幕宽度	布局策略	字体比例
手机竖屏	< 600px	单列垂直布局	1.0x
平板竖屏	600-840px	双列布局	1.2x
平板横屏	> 840px	三列布局	1.5x

布局特点说明：

- 手机端采用紧凑的单列垂直布局，信息层级清晰，适合小屏幕展示
- 平板端利用横向空间，采用多列并行布局，提高信息密度

传感器集成分析

传感器类型	应用场景	数据更新频率	精度
陀螺仪	设备姿态检测、游戏控制、AR应用	100Hz	0.01°
光线传感器	自动亮度调节、环境光感知、护眼模式	10Hz	1 lux

数据采集流程说明：权限检查 → 注册监听器 → 实时获取数据 → 更新UI → 页面销毁时取消监听

分布式能力分析

分布式数据管理原理：基于KV存储实现跨设备数据同步，通过鸿蒙软总线技术实现设备间通信，支持多设备实时数据共享。

核心优势：

- 无需手动配置网络连接，系统自动发现周边设备
- 支持离线数据同步，网络恢复后自动同步
- 数据同步过程对开发者透明，易于使用

问题与解决

问题描述	解决方案	解决结果
传感器权限申请失败	在module.json5中配置权限声明，使用动态权限请求API	✓ 已解决
多端布局错乱	使用断点系统和响应式组件，根据屏幕宽度动态调整布局	✓ 已解决
分布式数据同步延迟	优化同步策略，使用	✓ 已解决

问题描述	解决方案	解决结果
	SYNC_MODE_PUSH_PULL模式	
天气图标显示异常	使用Unicode表情符号替代图片资源，确保跨平台兼容	✓ 已解决

实验总结

实验收获

- 掌握了ArkUI声明式开发框架的核心概念和组件使用方法
- 学会了多端响应式布局设计，实现了手机/平板自适应
- 了解了传感器数据采集与实时展示方法
- 理解了鸿蒙分布式能力原理及跨设备数据同步机制
- 体验了AI辅助开发在鸿蒙应用开发中的实践应用，显著提高了开发效率

技术要点

- **ArkTS语言**：声明式UI开发，状态管理，自定义组件
- **响应式布局**：断点系统，自适应字体和间距
- **传感器API**：光线传感器、陀螺仪传感器的注册与监听
- **分布式能力**：KV存储、跨设备数据同步
- **AI辅助开发**：提示词工程、代码生成、调试优化

考核指标

考核项	评分标准	得分
功能完整性	天气应用功能完整，多端适配正常	25分
代码质量	代码结构清晰，注释完整	20分
多端适配	手机/平板界面自适应良好	15分

考核项	评分标准	得分
传感器集成	传感器数据采集正常	15分
分布式能力	分布式数据同步功能实现	15分
实验报告	报告结构完整，内容详实	10分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名：_____

日期：_____